Nexora: An AI-Powered Security Event Correlation and Incident Reconstruction Platform

Project Proposal for UCS503P

Submitted to: Nisha Thakur

Thapar Institute of Engineering and Technology

Apurv — 1024030791 Mohd. Noman — 1024030794
Shiven Bansal — 1024030799 Somraj Singha — 1024030795

August 15, 2026

1 Higher-order goal

This project aims to close the gap between raw anomaly detection and actionable security response by reconstructing scattered suspicious events into a single, explainable incident. Enterprise Security Operations Centres already do this at scale using correlation and SIEM tooling; the immediate engineering objective here is narrower and concrete: to build a smaller, validated, and explainable version of that correlation mechanism – one that detects, classifies, links, and reconstructs an attack chain from raw activity, rather than surfacing a flood of disconnected alerts.

2 Time-to-value

- The smallest useful product is a detector that flags a single suspicious event (for example, a login burst) and assigns it a risk score. This is demonstrable in the first iteration and is independently valuable even before correlation, classification, or a dashboard exist.
- We prioritise fast feedback by building the pipeline in explicit, testable stages – event generation, detection, classification, correlation – so each stage is independently verifiable before the next is layered on top.
- The work splits into parallel workstreams: the event/detection pipeline (ML), the correlation and reconstruction engine, and the dashboard frontend, allowing concurrent rather than serial progress.
- Success in the first iteration is defined by measurement, not feature count: anomaly detection must be validated against a real labelled dataset before correlation logic is layered on top of it.

3 Problem Statement

A conventional anomaly detector treats every suspicious event in isolation. An attacker rarely produces a single unusual action; a credential attack, for instance, typically manifests as a sequence – repeated failed logins, an eventual successful login from an unfamiliar source, access to a sensitive resource, and an abnor-mal data transfer. Presented separately, these appear as four unrelated, low-context alerts. This produces four recurring problems:

- Alert fragmentation: a single coherent attack is reported as multiple disconnected alerts, forcing an analyst to manually infer the relationship between them.
- Loss of narrative context: individual alerts carry a risk score but no story – there is no indication of what an attacker was doing, only that something looked unusual at a point in time.

- Circular validation: systems trained and evaluated purely on self-generated simulated attacks cannot demonstrate that they generalise to real intrusion behaviour.
- High cost of the remedy: correlation and incident-reconstruction capability exists in mature commer-cial SIEM and XDR platforms, but is either operationally heavy or priced for enterprise deployment, leaving smaller teams without an accessible equivalent.

Why this matters now (contextual relevance): as organisations instrument more of their infrastructure (logins, APIs, databases, file access, network calls), the volume of low-level anomaly alerts grows faster than analyst capacity to triage them individually. A layer that reconstructs related events into one incident, with evidence and severity attached, is therefore a practical necessity rather than a hypothetical convenience.

4 Proposed Solution ... Software Proposition

4.1 Overview

We propose Nexora, an AI-powered monitoring platform that observes organisational activity, detects and classifies suspicious events, and correlates related events into a single reconstructed security incident. Its components are:

- Event pipeline: a simulated organisation generating login, API, database, file, and network events, with an injectable attack generator layered on top of normal baseline traffic.
- Detection engine: an Isolation Forest model (optionally an autoencoder) that scores each event for anomalousness relative to learned normal behaviour.
- Classification engine: an ML classifier that labels a flagged event by attack type – brute force / credential stuffing, port scanning, abnormal data exfiltration, or API request bursts.
- Correlation engine: an entity-linked event graph, built over a sliding time window, that links events sharing a user, IP, or session, and matches the resulting sequence against predefined attack-chain patterns.
- Risk and severity scoring: a per-event risk score that aggregates across a correlated chain and re-ceives a severity boost when the chain matches a recognised attack pattern.
- Dashboard: a React interface presenting a live activity stream, an anomaly graph, an incident time-line, an evidence panel, and a recommended action.

4.2 Core workflow

- The simulator generates baseline organisational activity across login, API, database, file, and network channels, with attack scenarios injected on top.

- The detection engine scores each event; events above threshold are passed to the classification engine.
- The classifier labels the event by attack type and forwards it, with its score, to the correlation engine.
- The correlation engine adds the event as a node to an entity-linked graph, connecting it to related events sharing user, IP, or session identity within the active time window.
- The engine checks the resulting subgraph against predefined attack-chain patterns (for example, AUTH FAILURE BUR

→ SUCCESS LOGIN NEW SOURCE → PRIVILEGED RESOURCE ACCESS → LARGE DATA TRANSFER).

- On a pattern match, individual risk scores are aggregated and a severity boost is applied, producing a

single incident record.

- The dashboard renders the incident's timeline, evidence, severity, and a recommended action, reading from the backend over REST.

4.3 Pipeline summary:

Organisation Activity

|

Event Generation (Login / API / DB / File / Network)

|

Feature Extraction

|

Detection Engine (Anomaly Detection + Attack Classification)

| Suspicious Events

|

Correlation Engine (Event Graph + Attack-Chain Patterns)

| Incident Created

|

Dashboard (Timeline, Evidence, Severity, Recommended Action)

4.4 Operational constraints for a course-project setting

- The correlation engine is a hybrid graph-linking and pattern-matching system, not a learned causal-inference model. This is a deliberate scope decision: it is explainable, easier to implement and defend, and fully deterministic.
- The detection and classification models are trained and validated on a real, labelled intrusion dataset before being exercised against the live simulated environment, to avoid the circularity of training and testing on self-generated data alone.
- The scope is deliberately limited to three to four attack types, to keep the correlation and reconstruc-tion logic deep rather than building many shallow attack simulations.
- The system must run on a simulator-driven environment reproducible by an evaluator without requir-ing access to production infrastructure or live traffic.

5 Solution Approach ... Engineering focus

This is an engineering course project. The emphasis therefore falls on correct, explainable correlation and on validated detection accuracy, rather than on novel machine-learning research.

5.1 Detection and classification strategy ... non-research

- Anomaly detection: Isolation Forest is used because anomalous points are easier to isolate via ran-dom partitioning than normal points, giving a natural anomaly score without requiring a labelled anomaly class. An autoencoder is retained as an optional secondary detector.
- Attack classification: a supervised classifier (Scikit-learn) labels a flagged event into one of the scoped attack categories, using features extracted from the event stream (request rate, failure rate, payload size, destination sensitivity, and similar signals).
- Correlation, not causal inference: the correlation engine deliberately avoids trying to learn causal-ity between events. It instead relies on explicit entity relationships (same user, same IP, same ses-sion, temporal proximity) plus a library of predefined attack-chain patterns, matched against the event graph.
- Validation discipline: the detection and classification models are trained and evaluated on a real, la-belled intrusion dataset (CICIDS2017 or UNSW-NB15) using precision, recall, and F1, independently of the simulated environment used for the live demonstration.

5.2 Web architecture ... web-based solution

A three-tier architecture with a dedicated detection/correlation service:

- Frontend: React + TypeScript single-page application, communicating with the backend over REST and WebSockets for live updates.
- Backend: FastAPI, exposing endpoints for event ingestion, incident retrieval, and dashboard aggre-gates.
- Detection and correlation layer: Python services running the Isolation Forest / classifier models and the event-graph correlation engine.
- Database: PostgreSQL (or Supabase), holding raw events, classified alerts, incidents, and evidence records.
- Real-time channel: WebSockets, streaming the live activity feed and newly created incidents to the dashboard as they occur.

5.3 Request/event path:

Simulated Organisation

|

Event Generation –> Feature Extraction

|

Detection Engine (Isolation Forest / Autoencoder)

|

Attack Classifier (Scikit-learn)

|

Correlation Engine (Event Graph + Pattern Matching) –> Risk Scoring

|

Incident written –> PostgreSQL / Supabase

|

FastAPI (REST + WebSockets) –> React Dashboard

5.4 Technology selection

5.5 Layer Technology

Machine learning Python

ML algorithms Scikit-learn (Isolation Forest, classifier) Data processing Pandas

Backend FastAPI

Frontend React + TypeScript

Database PostgreSQL / Supabase Real-time updates WebSockets

5.6 CI/CD and fast delivery enabler

CI/CD will be used to:

- run backend unit tests and detection/classification model checks on every change;
- run a correlation-accuracy test suite against known attack-chain fixtures, so a regression in pattern matching fails the build rather than reaching the demo environment;
- produce repeatable deployment artefacts for staging;
- enable frequent, low-risk delivery of incremental modules (detection, then classification, then corre-lation, then dashboard).

6 Evaluation Criterion

Success is defined by instrumented measurement against a real dataset and a controlled simulated attack, not by feature presence.

6.1 Primary evaluation metric

Incident reconstruction accuracy (IRA): for a controlled sequence of injected attack events forming a known attack chain, the proportion of cases in which the correlation engine correctly links all related events into a single incident record, rather than reporting them as separate alerts or merging unrelated events.

- Target: correct single-incident reconstruction in at least 90% of injected attack-chain scenarios, across all scoped attack types.

- Attribution: computed by comparing the incidents produced by the correlation engine against the ground-truth chain used to generate the injected attack. Under-linking (missed related events) and over-linking (unrelated events merged) are both attributable to the correlation engine, not to the de-tection or classification stages.

6.2 Secondary evaluation metrics

- Detection quality (real dataset): precision, recall, and F1 of the Isolation Forest / classifier stage on CICIDS2017 or UNSW-NB15, evaluated independently of the simulated demo environment.
- Classification accuracy: proportion of flagged events correctly labelled by attack type, evaluated against the same real, labelled dataset.
- False incident rate: proportion of normal, non-attack activity in the simulated environment incor-rectly reconstructed into an incident. Target: kept low enough that the dashboard remains usable

rather than noisy.

- Dashboard freshness: lag between an event occurring in the simulator and its appearance in the live activity stream and, where applicable, in a reconstructed incident.
- Evidence completeness: proportion of a reconstructed incident's contributing events that are cor-rectly listed in its evidence panel.

6.3 Pilot validation plan ... high-level

- Train and evaluate the detection and classification models on CICIDS2017 or UNSW-NB15, reporting precision, recall, and F1 per attack type.
- Run the simulated organisation environment with baseline traffic, then inject each of the scoped attack scenarios in turn, recording whether the correlation engine reconstructs the expected single incident.
- Run a mixed session combining baseline traffic with two overlapping attack injections, to check that the correlation engine does not merge unrelated attacks into one incident.
- Walk through the dashboard's evidence panel and recommended action for each reconstructed incident against the known ground truth, as a qualitative check on explainability.

7 Scalability

The proposition should scale in both theoretical and practical senses.

- Scalable (theoretically): the design admits growth in event volume, entity count, and attack-pattern library size by:
- keeping the event graph windowed rather than unbounded, so correlation cost does not grow with the full history of the organisation;
- externalising attack-chain definitions as data rather than code, so new patterns can be added without modifying the correlation engine itself;
- separating the detection/classification stage from the correlation stage, so either can be scaled or re-trained independently;
- pre-aggregating dashboard data server-side rather than shipping raw event streams to the browser.
- Operationally deployable (practically): the system runs on commodity infrastructure:
- a managed PostgreSQL / Supabase instance for persistence;
- a containerised FastAPI backend, permitting horizontal replication of the detection and correla-tion services;
- static hosting of the React frontend;
- a CI/CD pipeline targeting a reproducible staging environment for evaluation.

8 Engine availability heuristic

A mature set of components exists for every layer of this proposition, which is what makes the scope realistic within a course timeline:

- Scikit-learn for Isolation Forest, the attack classifier, and standard evaluation metrics.
- Pandas for feature extraction and dataset preparation from CICIDS2017 / UNSW-NB15.
- FastAPI for the backend REST and WebSocket layer.
- PostgreSQL / Supabase for durable storage of events, alerts, and incidents.
- React + TypeScript for the dashboard frontend.
- Standard graph and windowing techniques for the entity-linked event graph, requiring no novel algo-rithmic research.

We use these mature components deliberately, so that the engineering effort is spent on correct correlation logic, validated detection, and a coherent incident narrative, rather than on reimplementing infrastructure.

9 Project scope and deliverables ... iteration-friendly

9.1 Initial deliverable ... first iteration

- Baseline event simulator generating login, API, database, file, and network events for a small simu-lated organisation.
- Isolation Forest anomaly detector, trained and validated on a real, labelled dataset.
- A single attack injector (brute-force / credential stuffing) exercised against the live simulator.
- Basic event and alert logging to PostgreSQL, sufficient to compute detection-quality metrics.
- CI: build, backend unit tests, and model validation checks on every push.

9.2 Subsequent deliverables

- Attack classifier covering the remaining scoped attack types (port scanning, exfiltration, API bursts).
- Correlation engine: entity-linked event graph with sliding time window and predefined attack-chain pattern matching.
- Risk and severity scoring, including the pattern-match severity boost.
- React dashboard: live activity stream, anomaly graph, incident timeline, and evidence panel.
- Recommended-action module attached to each reconstructed incident.
- (Stretch) Support for concurrent, overlapping attack scenarios in a single simulated session.
- (Stretch) Configurable attack-chain pattern library editable without code changes.

10 Risks and mitigations

- Overselling correlation as "AI": presenting the graph-linking and pattern-matching engine as a learned causal model would be inaccurate and hard to defend. Mitigation: describe it explicitly as a hybrid graph-linking and pattern-matching system, with anomaly detection and classification as the model-based components.
- Circular validation: training and testing purely on self-generated simulated attacks would not demon-strate real-world validity. Mitigation: validate detection and classification

exclusively against a real, labelled dataset (CICIDS2017 / UNSW-NB15), reserving the simulator for live demonstration only.

- Over-linking or under-linking in correlation: the event graph could merge unrelated events or fail to link related ones. Mitigation: bound the sliding time window explicitly, require matching entity attributes (user, IP, or session) before linking, and include mixed-session tests with overlapping attacks in the validation plan.
- Attack-type scope creep: adding attack scenarios beyond the scoped three to four types risks shallow coverage everywhere. Mitigation: fix the attack-type list early and treat additional types as stretch

goals only.

- Dashboard overload: presenting excessive raw event detail could make the incident view harder to read than the alerts it replaces. Mitigation: keep the dashboard focused on the reconstructed incident narrative – timeline, evidence, severity, recommended action – with raw events available on drill-down only.

11 Summary

This project proposes Nexora, an AI-powered security monitoring platform that moves from flagging indi-vidual outliers to reconstructing them into one explainable security incident. It detects suspicious events with an Isolation Forest model, classifies them by attack type, and correlates related events using an entity-linked event graph matched against predefined attack-chain patterns – a deliberately explainable and deterministic design rather than a learned causal model. Detection and classification are validated against a real, labelled intrusion dataset (CICIDS2017 or UNSW-NB15), while a simulated organisational environment drives live attack injection and incident reconstruction for demonstration. A React dashboard presents the resulting in-cident's timeline, evidence, severity, and recommended action.

In one line: Detect → Classify → Correlate→ Reconstruct → Explain.